

■ Blood Bond — ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■

Generated from *BloodBond-Project-Documentation.md*. Covers the full backend (.NET) and frontend (React) implementation, every API endpoint, Postman scenarios, role permissions, enums, deployment, and troubleshooting.

■ Blood Bond — ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■

■■■■■ ■■■■■■■■■■

- [■■■■ ■■■■ ■■■■ ■■■■■■■■](#)
- [■■■■■■ ■■■■■■■■ \(Tech Stack\)](#)
- [■■■ Backend — ■■■■ ■■■■■■■■](#)
- [■■■ Frontend — ■■■■ ■■■■■■■■](#)
- [■■■■■ ■■ ■■■■ API Endpoints ■■ ■■■■■■](#)
- [■■■■■■■■■■ ■■■■■■■■■■ ■■■■ Postman](#)
- [■■■ Roles ■■■■■■■■■■](#)
- [■■■ Enums ■■■■■■■■■■](#)
- [■■■ Deployment ■■■■■■](#)
- [■■ ■■■■■■■■ ■■■■■■■■ \(Troubleshooting\)](#)

1. ■■■■■ ■■■■■ ■■■■ ■■■■■■■■■■

Blood Bond ■■ ■■■■ ■■■■■■ ■■■■■■ ■■■■■■ ■■■■ ■■■■:

- ■■■■■■■■■■ (Donors / Users)
- ■■■■ ■■■■ (Blood Banks) ■■ ■■■■■ ■■■ ■■■■■■
- ■■■■■■ ■■■■ ■■■■■■ ■■ (Blood Requests)
- ■■■■■■ ■■■■■■■■■■ (Blood Bank Managers) ■■■■ ■■■■■■ ■■■■■■
- ■■■■ Admins ■■■■ ■■■■■■■■ ■■■ ■■■■■■ ■■■■ donations ■■■■ users

■■■■■■■■ ■■■■■■■■:

- ■ Authentication (Login / Register / Forgot Password)
- ■ User Roles (Admin / BloodBankManager / User)
- ■ Blood Banks Management (Create / Edit / Approve / Reject / Inventory)
- ■ Blood Requests (Patient requests → Manager fulfills)

- ■ Donations (Schedule → Approve → Complete → Award Points)
- ■ Monetary Donations (Stripe Checkout)
- ■ Events (Blood drives / Community meetups)
- ■ Badges (Gamification / Leaderboard)
- ■ Ratings & Reviews

2. ■■■■■■ ■■■■■■ (Tech Stack)

Backend:

Technology	Version	Purpose
ASP.NET Core	.NET 9.0	Web API framework
Entity Framework Core	9.0	ORM for database
SQL Server	Latest	Database (on MonsterASP)
ASP.NET Identity	Built-in	User management
JWT Bearer	Built-in	Authentication tokens
Stripe.net	Latest	Payment processing
BCrypt	Built-in	Password hashing

Frontend:

Technology	Version	Purpose
React	18.3	UI framework
Vite	5.4	Build tool & dev server
React Router	6.27	Client-side routing
React Bootstrap	2.10	UI components
Bootstrap	5.3	CSS framework
Axios	1.7	HTTP client
React Hot Toast	2.4	Notifications

Infrastructure:

- **Backend Host:** MonsterASP (<https://blood-bond.runasp.net>)
- **Database Host:** MonsterASP Database ([db65353](#))
- **Frontend Host:** Vercel (deployed via Vercel CLI / Git)
- **Payment:** Stripe (Test mode: [sk_test_51T0l82B...](#))

3. Backend —

```

BloodBond/
■■■ BloodBond.sln # Solution file
■■■ BloodBond/ # Main API project
■ ■■■ Program.cs # Entry point
■ ■■■ appsettings.json # Config (empty for secrets)
■ ■■■ Properties/launchSettings.json # Dev environment = "Development"
■ ■■■ Controllers/ # API endpoints
■ ■ ■■■ AccountController.cs # Auth (login/register/etc)
■ ■ ■■■ AdminController.cs # Admin operations
■ ■ ■■■ BloodBanksController.cs # Blood banks CRUD
■ ■ ■■■ BloodRequestsController.cs # Patient requests
■ ■ ■■■ DonationsController.cs # Donor schedules
■ ■ ■■■ EligibilityController.cs # Eligibility check
■ ■ ■■■ EventsController.cs # Blood drives / events
■ ■ ■■■ MonetaryDonationsController.cs # Stripe payments
■ ■ ■■■ RatingsController.cs # Bank reviews
■ ■ ■■■ BadgesController.cs # Gamification
■ ■ ■■■ DebugController.cs # Diagnostic endpoint
■ ■■■ Extensinos/ # Service registration
■ ■ ■■■ ApplicationServicesExtensions.cs # AddApplicationServices()
■ ■ ■■■ CorsPolicyExtensions.cs # AddCorsPolicy()
■ ■ ■■■ IdentityExtensions.cs # AddIdentityServices()
■ ■ ■■■ BuilderExtensions.cs # Middleware pipeline
■ ■■■ Middleware/
■ ■■■ GlobalExceptionHandler.cs # Maps exceptions to HTTP codes
■■■ BloodBond.BLL/ # Business Logic Layer
■ ■■■ Service/
■ ■ ■■■ BloodBankService.cs # + IsAdminOrManager()
■ ■ ■■■ BloodRequestService.cs # + GetForBankAsync()
■ ■ ■■■ DonationService.cs
■ ■ ■■■ EligibilityService.cs
■ ■ ■■■ EventService.cs # BloodDriveEventService
■ ■ ■■■ MonetaryDonationService.cs # Stripe Checkout + GetByBankAsync + GetAllAsync
■ ■ ■■■ RatingService.cs
■ ■ ■■■ BadgeService.cs
■ ■ ■■■ UserManagementService.cs # Admin operations
■ ■ ■■■ IStripeSettings.cs # StripeSettings class
■ ■ ■■■ ...
■ ■■■ DTO/
■ ■ ■■■ Request/ # Input DTOs
■ ■ ■ ■■■ BloodDriveEventRequest.cs # { BloodBankId, Title, Location, EventDate,
Description?, Capacity }
■ ■ ■ ■■■ BloodRequestRequest.cs # { BloodType, UnitsNeeded, UrgencyLevel, City, Notes
}
■ ■ ■ ■■■ DonationRequest.cs # { BloodBankId, ScheduledDate, Notes? }
■ ■ ■ ■■■ MonetaryDonationRequest.cs # { Amount, Currency, BloodBankId? }
■ ■ ■ ■■■ ChangeRoleRequest.cs # { Role }
■ ■ ■■■ Response/ # Output DTOs
■ ■ ■■■ BloodDriveEventResponse.cs # { Id, BloodBankId, BloodBankName, Title, Location,
EventDate, Capacity, RegisteredCount }
■ ■ ■■■ BloodRequestResponse.cs # { Id, BloodType, UnitsNeeded, UrgencyLevel, Status,
City }
■ ■ ■■■ DonationResponse.cs # { Id, BloodBankId, BloodBankName, ScheduledDate, Status,
UnitsDonated }
■ ■ ■■■ MonetaryDonationResponse.cs # { Id, DonorId, DonorName, BloodBankId,
BloodBankName, Amount, Currency, Status }
■ ■ ■■■ PaymentIntentResponse.cs # { ClientSecret, PaymentIntentId, CheckoutUrl, IsMock,
Amount, Currency }
■ ■ ■■■ UserListResponse.cs # { Id, FullName, Email, IsBlocked, Roles[] }
■ ■ ■■■ EligibilityResponse.cs
■ ■■■ Service/
■ ■■■ IDonationService.cs

```



```

■■■■ context/
■ ■■■■ AuthContext.jsx # user, token, role, login(), register(), logout()
■ ■■■■ constants.js # BLOOD_TYPES, URGENCY_LEVELS, ROLES, status enums
■■■■ pages/
■ ■■■■ HomePage.jsx # Public landing
■ ■■■■ LoginPage.jsx # /login
■ ■■■■ RegisterPage.jsx # /register
■ ■■■■ ForgotPasswordPage.jsx # /forgot
■ ■■■■ DashboardPage.jsx # /dashboard (role-based: Admin/Manager/Donor)
■ ■■■■ BloodBanksPage.jsx # /blood-banks
■ ■■■■ BloodRequestsPage.jsx # /requests (manager view by default)
■ ■■■■ DonationsPage.jsx # /donations
■ ■■■■ MonetaryDonationsPage.jsx # /monetary (with mock mode fallback)
■ ■■■■ AdminMonetaryPage.jsx # /admin/monetary (admin: all donations)
■ ■■■■ EligibilityPage.jsx # /eligibility
■ ■■■■ EventsPage.jsx # /events (3 tabs: Upcoming / Mine / Manage)
■ ■■■■ BadgesPage.jsx # /badges (leaderboard)
■ ■■■■ RatingsPage.jsx # /ratings
■ ■■■■ ProfilePage.jsx # /profile
■ ■■■■ AdminUsersPage.jsx # /admin/users
■ ■■■■ AdminDiagnosticPage.jsx # /admin/diagnostic (status of all features)
■ ■■■■ ContactUsPage.jsx # /contact (mailto fallback)
■ ■■■■ ForbiddenPage.jsx # /forbidden (403)
■ ■■■■ NotFoundPage.jsx # /404
■■■■ utils/
■■■■ error.js # apiErrorMessage(err) → friendly message

```

5. ■■■■■■ ■■ ■■■■ API Endpoints ■■ ■■■■■■

■ Auth Endpoints (/api/Account/*)

POST /api/Account/Register

- **Auth:** Public (AllowAnonymous)
- **Body:**

```

{
  "fullName": "John Doe",
  "email": "john@example.com",
  "password": "Pass@1234",
  "confirmPassword": "Pass@1234"
}

```

- **Response:** 200 OK (no body) | 400 Bad Request if validation fails
- **C#:** AccountController.Register

POST /api/Account/login

- **Auth:** Public
- **Body:**

```

{
  "email": "admin@bloodbond.com",
  "password": "Admin@123456"
}

```

- **Response:** 200 OK with token payload:

```
{
  "userId": "e5f3abfe-f505-42a4-b8b6-9f4102ff6c15",
  "email": "admin@bloodbond.com",
  "fullName": "System Admin",
  "roles": ["Admin"],
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expiresAt": "2026-09-14T09:19:04.0566991Z"
}
```

- **Notes:** Token is a JWT. Frontend stores in `localStorage.bb_token`.

GET /api/Account/me

- **Auth:** Required (any role)
- **Headers:** `Authorization: Bearer <token>`
- **Response:** `200 OK` with user info | `401 Unauthorized` if invalid token

POST /api/Account/forgot-password

- **Auth:** Public
- **Body:** `{ "email": "user@example.com" }`
- **Response:** `200 OK` + email containing reset token

POST /api/Account/reset-password

- **Auth:** Public
- **Body:**

```
{
  "email": "user@example.com",
  "token": "CfDJ8KOC0RNYt9JFoSbYJxYc5gP2JG6u3JqTSriy...",
  "newPassword": "NewPass@123",
  "confirmPassword": "NewPass@123"
}
```

■ Admin Endpoints (/api/admin/*)

POST /api/admin/register-first

- **Auth:** Public (only works once — refuses if any admin exists)
- **Body:**

```
{
  "secretKey": "dev-key-12345-abcde",
  "fullName": "Bootstrap Admin",
  "email": "bootstrap.admin@bloodbond.com",
  "password": "Bootstrap@2026"
}
```

POST /api/admin/create

- **Auth:** Admin only
- **Body:**

```
{
  "fullName": "Test Manager",
  "email": "test.manager@example.com",
  "password": "Manager@123",
  "role": "BloodBankManager"
}
```

- **role:** `User` | `BloodBankManager` | `Admin`

`POST /api/admin/change-password`

- **Auth:** Admin only

- **Body:**

```
{
  "currentPassword": "Admin@123456",
  "newPassword": "NewAdmin@123",
  "confirmPassword": "NewAdmin@123"
}
```

`GET /api/admin/users`

- **Auth:** Admin only

- **Response:** Array of `UserListResponse`:

```
[
  {
    "id": "e5f3abfe-f505-42a4-b8b6-9f4102ff6c15",
    "fullName": "System Admin",
    "email": "admin@bloodbond.com",
    "phoneNumber": "123456",
    "isBlocked": false,
    "createdAt": "2026-08-01T00:00:00Z",
    "roles": ["Admin"]
  }
]
```

`GET /api/admin/users/{id}`

- **Auth:** Admin only

- **Response:** Single `UserListResponse`

`PATCH /api/admin/users/{id}/block`

- **Auth:** Admin only

- **Response:** Updated user

`PATCH /api/admin/users/{id}/unblock`

- **Auth:** Admin only

`PATCH /api/admin/users/{id}/role`

- **Auth:** Admin only

- **Body:** `{ "role": "User" }` (or `BloodBankManager`, `Admin`)

- **Response:** Updated user

`GET /api/admin/analytics`

- **Auth:** Admin only

- **Response:**

```
{
  "Users": { "Total": 6 },
  "BloodBanks": { "Total": 1, "Verified": 1, "Pending": 0 },
  "Requests": { "Total": 5, "Pending": 4, "Fulfilled": 0, "CriticalPending": 0 },
}
```

```
"Donations": { "Total": 0, "Completed": 0 },
"Monetary": { "TotalDonatedUSD": 0 },
"Inventory": { "LowStockItems": 0 },
"BloodTypeDistribution": []
}
```

■ Blood Banks Endpoints (/api/bloodbanks/*)

GET /api/bloodbanks

- **Auth:** Public (AllowAnonymous)
- **Response:** Array of **BloodBankResponse**

GET /api/bloodbanks/verified

- **Auth:** Public
- **Response:** Array (only Verified banks)

GET /api/bloodbanks/{id}

- **Auth:** Public
- **Response:** Single **BloodBankResponse**

POST /api/bloodbanks

- **Auth:** User, Admin
- **Body:**

```
{
  "name": "Ramallah Central Blood Bank",
  "cityAddress": "Main Street, Ramallah",
  "latitude": 31.9038,
  "longitude": 35.2030,
  "contactPhone": "+970599000000"
}
```

- **Behavior:** Creates a bank with status = **Pending**. Auto-promotes User → **BloodBankManager**.

GET /api/bloodbanks/mine

- **Auth:** BloodBankManager, Admin
- **Response:** The bank managed by current user

PUT /api/bloodbanks/{id}

- **Auth:** BloodBankManager, Admin (with **IsAdminOrManager** check)
- **Body:** Same as POST
- **Backend fix:** **BloodBankService.IsAdminOrManager()** allows admins to edit any bank.

PUT /api/bloodbanks/{id}/inventory

- **Auth:** BloodBankManager, Admin
- **Body:**

```
[
  { "bloodType": 0, "unitsAvailable": 15 },
  { "bloodType": 1, "unitsAvailable": 8 },
]
```

```
{ "bloodType": 6, "unitsAvailable": 22 }
]
```

- **Backend fix:** `SetInventoryAsync` uses **upsert** (find by BloodBankId+BloodType, update if exists, insert otherwise). No more duplicate key errors.

PATCH /api/bloodbanks/{id}/approve

- **Auth:** Admin only
- **Response:** Bank with status = **Verified**

PATCH /api/bloodbanks/{id}/reject

- **Auth:** Admin only
- **Response:** Bank with status = **Rejected**

GET /api/bloodbanks/low-stock

- **Auth:** Public
- **Response:** Array of **BloodInventoryResponse** where units < 5

■ Blood Requests Endpoints (/api/bloodrequests/*)

POST /api/bloodrequests

- **Auth:** Required (any user)
- **Body:**

```
{
  "bloodType": 0,
  "unitsNeeded": 2,
  "urgencyLevel": 2,
  "city": "Ramallah",
  "notes": "Urgent surgery needed"
}
```

- **Behavior:** Creates request with status = **Pending**. Auto-notifies compatible donors.

GET /api/bloodrequests/mine

- **Auth:** Required
- **Response:** Requests created by current user

GET /api/bloodrequests/active?city=Ramallah

- **Auth:** Public
- **Query:** **city** (required)
- **Response:** Active requests (Pending or InProgress) in the city

GET /api/bloodrequests/by-bank/{bankId}

- **Auth:** BloodBankManager, Admin
- **Response:** Active requests in the city of the bank

GET /api/bloodrequests/{id}

- **Auth:** Required

PATCH /api/bloodrequests/{id}/cancel

- **Auth:** Required (owner only)
- **Response:** Request with status = **Cancelled**

PATCH /api/bloodrequests/{id}/fulfill

- **Auth:** BloodBankManager, Admin
- **Response:** Request with status = **Fulfilled**

POST /api/bloodrequests/{id}/notify

- **Auth:** Admin, BloodBankManager
- **Response:** { "notified": 5 } (count of compatible donors notified)

■ Donations Endpoints (/api/donations/*)

POST /api/donations

- **Auth:** Required (must pass eligibility)
- **Body:**

```
{
  "bloodBankId": 1,
  "scheduledDate": "2026-12-15T10:00:00",
  "notes": "First time donor"
}
```

GET /api/donations/mine

- **Auth:** Required
- **Response:** My donations (any status)

GET /api/donations/by-bank/{bankId}

- **Auth:** BloodBankManager, Admin
- **Response:** Donations to that bank

GET /api/donations/{id}

PATCH /api/donations/{id}/cancel

- **Auth:** Required (owner)

PATCH /api/donations/{id}/approve

- **Auth:** BloodBankManager, Admin
- **Status change:** **Scheduled** → **Approved**

PATCH /api/donations/{id}/reject

- **Auth:** BloodBankManager, Admin
- **Status change:** **Scheduled** → **Rejected**

PATCH /api/donations/{id}/complete

- **Auth:** BloodBankManager, Admin
- **Body:** { "unitsDonated": 1, "notes": "Successful" }
- **Status change:** → **Completed**. Updates inventory. Awards points to donor.

■ Eligibility Endpoints (/api/eligibility/*)

POST /api/eligibility

- **Auth:** Required
- **Body:**

```
{
  "weight": 75,
  "age": 28,
  "hasChronicDisease": false,
  "lastSurgeryDate": null
}
```

- **Response:** { "passed": true, "reasons": [] } or { "passed": false, "reasons": ["..."] }

GET /api/eligibility/latest

- **Auth:** Required
- **Response:** Latest eligibility check result

■ Events Endpoints (/api/events/*)

GET /api/events/upcoming

- **Auth:** Public
- **Response:** Upcoming events list

GET /api/events/mine

- **Auth:** Required
- **Response:** Events I'm registered for

GET /api/events/by-bank/{bankId}

- **Auth:** BloodBankManager, Admin

POST /api/events/{id}/register

- **Auth:** Required
- **Body:** Empty (uses current user)

POST /api/events/{id}/cancel

- **Auth:** Required (cancel RSVP)

POST /api/events/{id}/checkin

- **Auth:** Required

GET /api/events/{id}/attendees

- **Auth:** BloodBankManager, Admin

POST /api/events / PUT /api/events/{id} / DELETE /api/events/{id}

- **Auth:** BloodBankManager, Admin

- **Body:**

```
{
  "bloodBankId": 1,
  "title": "Ramallah Blood Drive",
  "location": "Al-Watani Hospital",
  "eventDate": "2026-12-15T10:00:00",
  "description": "Open day, free t-shirts",
  "capacity": 80
}
```

■ Monetary Donations Endpoints (/api/monetarydonations/*)

POST /api/monetarydonations/create-intent

- **Auth:** Required

- **Body:**

```
{
  "amount": 25.00,
  "currency": "usd",
  "bloodBankId": 1
}
```

- **Response** (real Stripe):

```
{
  "clientSecret": "cs_test_...",
  "paymentIntentId": "cs_test_...",
  "checkoutUrl": "https://checkout.stripe.com/c/pay/cs_test_...",
  "sessionId": "cs_test_...",
  "amount": 25.00,
  "currency": "usd",
  "status": "checkout_created",
  "isMock": false
}
```

- **Response** (mock mode — no Stripe key):

```
{
  "clientSecret": "pi_mock_xxx_secret_mock",
  "paymentIntentId": "pi_mock_xxx",
  "isMock": true,
  "status": "requires_payment_method"
}
```

POST /api/monetarydonations/webhook

- **Auth:** Public (called by Stripe)
- **Behavior:** Stripe sends `payment_intent.succeeded` → updates donation status to **Succeeded**

POST

/api/monetarydonations/confirm?paymentIntentId=X&status=Succeeded

- **Auth:** Public (used for manual confirmation or testing)

GET /api/monetarydonations/mine

- **Auth:** Required
- **Response:** My donations

GET /api/monetarydonations/total/mine

- **Auth:** Required
- **Response:** { "total": 25, "currency": "USD" }

GET /api/monetarydonations/by-bank/{bankId}

- **Auth:** BloodBankManager, Admin
- **Response:** { "total": 100, "currency": "USD" } (total donations to that bank)

GET /api/monetarydonations/by-bank-detail/{bankId}

- **Auth:** BloodBankManager, Admin
- **Response:** Array of donations for that bank

GET /api/monetarydonations/all

- **Auth:** Admin only
- **Response:** All donations across the system

■ Ratings Endpoints (/api/ratings/*)

POST /api/ratings

- **Auth:** Required
- **Body:**

```
{
  "bloodBankId": 1,
  "rating": 5,
  "comment": "Excellent service!"
}
```

GET /api/ratings/by-bank/{bankId}

GET /api/ratings/mine/{bankId}

GET /api/ratings/stats/{bankId}

■ Badges Endpoints (/api/badges/*)

GET /api/badges

- **Auth:** Required
- **Response:** All available badges

GET /api/badges/mine

- **Auth:** Required
- **Response:** Badges earned by current user

```
# GET /api/badges/me/rank
```

- **Auth:** Required
- **Response:** { "rank": 5, "points": 120 }

```
# GET /api/badges/leaderboard?top=10
```

- **Auth:** Required

...

6. ■■■■■■■■■■ ■■■■■■■■■■ ■■■■ Postman

Setup :

- **Login** as admin →  token
- Environment variable: **token** = <paste-token>
- Environment variable: **baseUrl** = <https://blood-bond.runasp.net>

1: Admin

```

1. POST /api/admin/create
- Headers: Authorization: Bearer {{token}}
- Body: { "fullName": "Test Manager", "email": "test@bb.com", "password": "Test@1234",
"role": "BloodBankManager" }
→ ■■■■ User ■■■■ ■■■■ BloodBankManager

2. GET /api/admin/users
→ ■■■■ ■■ ■■■■ users + roles[]

3. PATCH /api/admin/users/{id}/role
- Body: { "role": "Admin" }
→ ■■■■ ■■■■ role

4. PATCH /api/admin/users/{id}/block
→ ■■■■ ■■■■ user

5. GET /api/admin/analytics
→ ■■■■ ■■■■■■■■ ■■■■■■■■

```

2: Blood Bank flow

1. User **■■■■■**: POST /api/Account/register
→ Account **■■■■■** + Token
2. User **■■■■■ ■■■■**: POST /api/bloodbanks
- Body: { "name": "...", "cityAddress": "..." }
→ Bank **■■■** status = Pending, User promoted to BloodBankManager
3. Admin **■■■■■■■**: PATCH /api/bloodbanks/{id}/approve
→ Bank.status = Verified
4. Manager **■■■■■ ■■■■** inventory: PUT /api/bloodbanks/{id}/inventory
- Body: [{ "bloodType": 0, "unitsAvailable": 10 }, ...]
→ Inventory **■■■■■■■**
5. Admin **■■■■■ ■■■■ ■■■■■■ ■■■■■■** Edit/Inventory **■■■■ ■■■■**

3: Patient + Manager flow

```

1. Patient ■■■■: POST /api/Account/register
2. Patient ■■■■ ■■: POST /api/bloodrequests
- Body: { "bloodType": 0, "unitsNeeded": 2, "urgencyLevel": 2, "city": "Ramallah" }
→ Request.Status = Pending
3. Manager ■■■■: GET /api/bloodrequests/by-bank/1
→ ■■■■ ■■■■■■■■ ■■■■■■
4. Manager ■■■■: PATCH /api/bloodrequests/{id}/fulfill
→ Status = Fulfilled

```

■■■■■■■■■■ 4: Donation flow ■■■■

```

1. Donor ■■■■ eligibility: POST /api/eligibility
- Body: { "weight": 75, "age": 28, "hasChronicDisease": false }
→ { passed: true }

2. Donor ■■■■: POST /api/donations
- Body: { "bloodBankId": 1, "scheduledDate": "2026-12-15T10:00:00" }
→ Status = Scheduled

3. Manager ■■■■: PATCH /api/donations/{id}/approve
→ Status = Approved

4. Manager ■■■■: PATCH /api/donations/{id}/complete
- Body: { "unitsDonated": 1, "notes": "Successful" }
→ Status = Completed, inventory updated, points awarded

```

■■■■■■■■■■ 5: Monetary Donation flow

```

1. Donor ■■■■ donate: POST /api/monetarydonations/create-intent
- Body: { "amount": 25, "currency": "usd", "bloodBankId": 1 }
→ Response: { checkoutUrl, ... } (real Stripe) ■■ { isMock: true } (mock)

2. (Real Stripe) User ■■■■ ■■■■■■■■ ■■■■■■ ■■ checkout.stripe.com
3. (Mock mode) Frontend ■■■■ modal ■■■■ test card form
4. Stripe webhook ■■■■ status: Succeeded
5. Manager ■■■■: GET /api/monetarydonations/by-bank-detail/1
→ ■■■■ donations ■■■■

```

7. ■■■■ Roles ■■■■■■■■■■

Role	■■■■■■■■■■
User (Donor)	Register/Login, Schedule donation, Create blood request, Donate money, Register for events, View badges/rank
BloodBankManager	■■ ■■ User + Edit/Inventory ■■ bank■, Approve/Reject donations/requests, Create events, View bank's donations
Admin	■■ ■■ Manager + Manage users, Approve/Reject banks, View all donations, View analytics

■■■ Admin Override:

■■■ **BloodBankService** ■■■ method **isAdminOrManager()**:

```

private bool IsAdminOrManager(string userId, string? managerId)
{

```

```

if (managerId == userId) return true; // Same manager
var role = _http.HttpContext?.User?.FindFirst(ClaimTypes.Role)?.Value;
return role == "Admin"; // Admins can override
}

```

■■■■ GlobalExceptionHandler ■■■■■■:

- UnauthorizedAccessException → 403 Forbidden
- KeyNotFoundException → 404 Not Found
- InvalidOperationException → 409 Conflict
- Exception → 500 Internal Server Error

8. ■■■■ Enums ■■■■■■

BloodType (0..7):

```

0 = A+  4 = AB+
1 = A-  5 = AB-
2 = B+  6 = O+
3 = B-  7 = O-

```

UrgencyLevel (0..3):

```

0 = Low (secondary)
1 = Normal (info)
2 = High (warning)
3 = Critical (danger)

```

DonationStatus (0..4):

```

0 = Scheduled (warning) - awaiting bank approval
1 = Approved (info) - approved, awaiting donation day
2 = Rejected (danger) - rejected by bank
3 = Completed (success) - donated successfully
4 = Cancelled (secondary) - cancelled by donor

```

RequestStatus (0..4):

```

0 = Pending (warning)
1 = InProgress (info)
2 = Fulfilled (success)
3 = Cancelled (secondary)
4 = Expired (danger)

```

BloodBankStatus (0..3):

```

0 = Pending (warning)
1 = Verified (success)
2 = Rejected (danger)
3 = Suspended (secondary)

```

CheckInStatus (0..3):

```

0 = Registered
1 = CheckedIn
2 = Cancelled
3 = NoShow

```

9. ■■■■ Deployment ■■■■■■

Backend ■■■■ MonsterASP:

→ **Publish:**

```
cd D:\BackEnd\testproject\Blood-Bond
dotnet publish -c Release -o ./publish
```

→ **Upload ■■■■■■ publish/ ■■■ MonsterASP ■■■ FTP**

→ **Environment Variables (■■■■■):**

```
ASPNETCORE_ENVIRONMENT = Production
Stripe__SecretKey = sk_test_51T0l82B...
Stripe__PublishableKey = pk_test_51T0l82B...
Stripe__WebhookSecret = whsec_...
Stripe__SuccessUrl =
https://your-app.vercel.app/payment-success?session_id={CHECKOUT_SESSION_ID}
Stripe__CancelUrl = https://your-app.vercel.app/payment-cancel
```

→ **Stripe Webhook (■■■ dashboard.stripe.com):**

- URL: <https://blood-bond.runasp.net/api/monetarydonations/webhook>
- Events: [payment_intent.succeeded](#), [payment_intent.payment_failed](#)

Frontend ■■■■ Vercel:

→ **Vercel Environment Variable:**

```
VITE_API_BASE_URL = https://blood-bond.runasp.net
```

→ **Deploy:**

- ■■■■■ [BloodBond.Web/dist/](#) ■■■ ■■■■■ ■■■ Git
- **Redeploy ■■■ Clear build cache**

Local Development:

```
# Backend (■■■ secrets)
$env:ASPNETCORE_ENVIRONMENT = 'Development'
cd D:\BackEnd\testproject\Blood-Bond\BloodBond
dotnet run

# Frontend
cd D:\BackEnd\testproject\Blood-Bond\BloodBond.Web
npm install
npm run dev
```

Secrets (local dev):

■■■■: [%APPDATA%\microsoft\UserSecrets\ff37ed95-ec97-4de3-ab64-7faab9a587d2\secrets.json](#)

```
{
  "Jwt": {
    "Key": "...",
    "Issuer": "BloodBondApi",
    "Audience": "BloodBondClient",
    "DurationInDays": "7"
  },
  "AdminBootstrap": {
    "SecretKey": "dev-key-12345-abcde"
  },
}
```

Stripe Checkout locale

■■■■■: ■■■ Stripe locale ■■■■ ■■■ browser ■■■■: ■■■ default ■■ Stripe ■■■■ ■■■■■■ ■■
Stripe Checkout settings

■■■■■ ■■■■:

■■■■■	■■■ URL	■■■ Method	■■■ Auth
Login	/api/Account/login	POST	Public
Register	/api/Account/Register	POST	Public
Current User	/api/Account/me	GET	Auth
List Users	/api/admin/users	GET	Admin
Change Role	/api/admin/users/{id}/role	PATCH	Admin
Block User	/api/admin/users/{id}/block	PATCH	Admin
Analytics	/api/admin/analytics	GET	Admin
List Banks	/api/bloodbanks	GET	Public
Create Bank	/api/bloodbanks	POST	User/Admin
Approve Bank	/api/bloodbanks/{id}/approve	PATCH	Admin
Update Inventory	/api/bloodbanks/{id}/inventory	PUT	Manager/Admin
Create Request	/api/bloodrequests	POST	User
Fulfill Request	/api/bloodrequests/{id}/fulfill	PATCH	Manager/Admin
Bank Requests	/api/bloodrequests/by-bank/{bankId}	GET	Manager/Admin
Schedule Donation	/api/donations	POST	User
Approve Donation	/api/donations/{id}/approve	PATCH	Manager/Admin
Complete Donation	/api/donations/{id}/complete	PATCH	Manager/Admin
Eligibility Check	/api/eligibility	POST	User
Upcoming Events	/api/events/upcoming	GET	Public

Endpoint	URL	Method	Auth
Register for Event	/api/events/{id}/register	POST	User
Stripe Donation	/api/monetarydonations/create-intent	POST	User
Bank Donations	/api/monetarydonations/by-bank-detail/{bankId}	GET	Manager/Admin
All Donations	/api/monetarydonations/all	GET	Admin
Add Rating	/api/ratings	POST	User
My Badges	/api/badges/mine	GET	User
Leaderboard	/api/badges/leaderboard	GET	User

!!! If you see any errors, please report them.